

```
1 int a=10;
```

```
2 a=20;
```

```
3 a=30;
```

```
4
```

```
5 Arrays:
```

```
6 disadv:
```

```
7 1. fixed in size
```

```
8 2. homoge.....
```

```
9 3. non underlying DS |
```

```
0
```

```
1 class Object{
```

```
2
```

```
3 }
```

```
4
```

```
5 class System
```

```
extends
```

```
Object{
```

```
6
```

```
7 }
```

```
8 -----
```

```
9 class MyJava{
```

```
10
```

```
11 }
```

```
12
```

Why Collection Framework

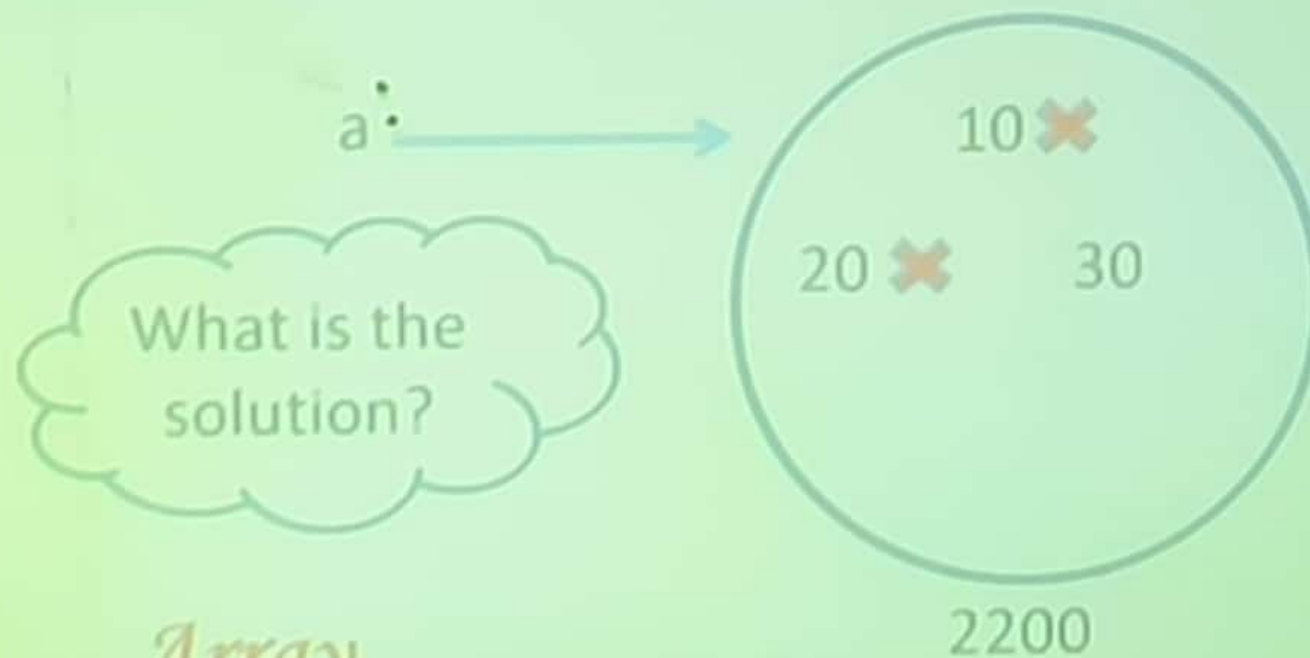
What is variable?

Variable is a name given to memory location or storage area

```
int a=10;
```

```
a=20;
```

```
a=30;
```



Array

Why Collection Framework

What is variable?

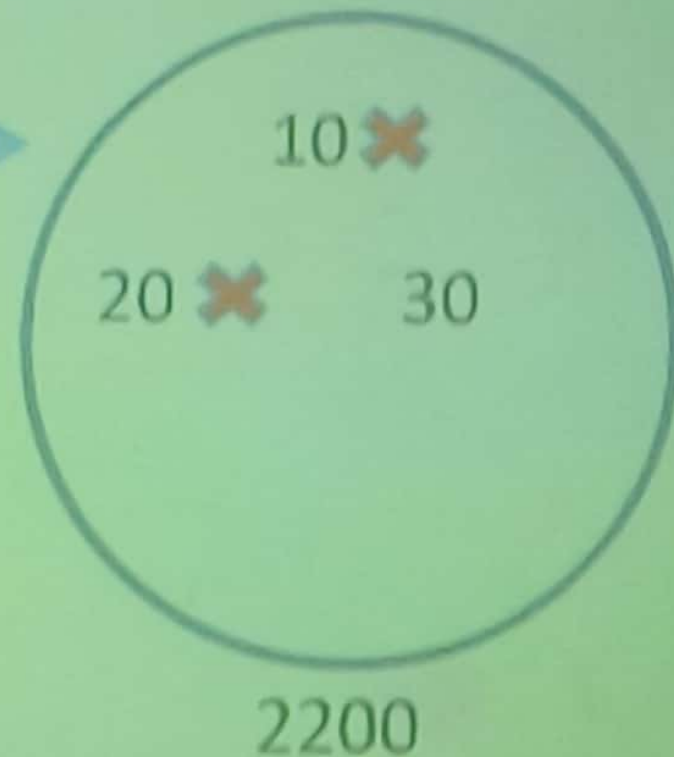
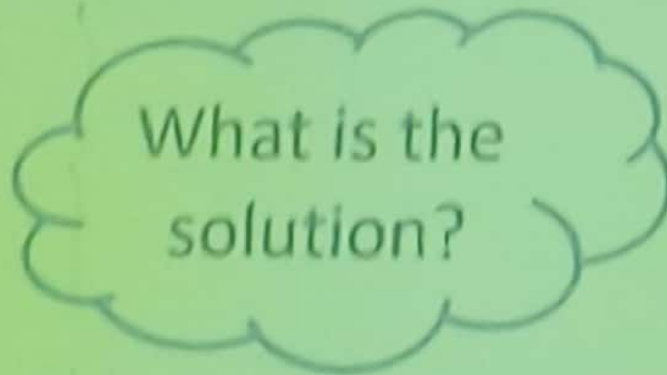
Variable is a name given to memory location or storage area

```
int a=10;
```

```
a=20;
```

```
a=30;
```

a →



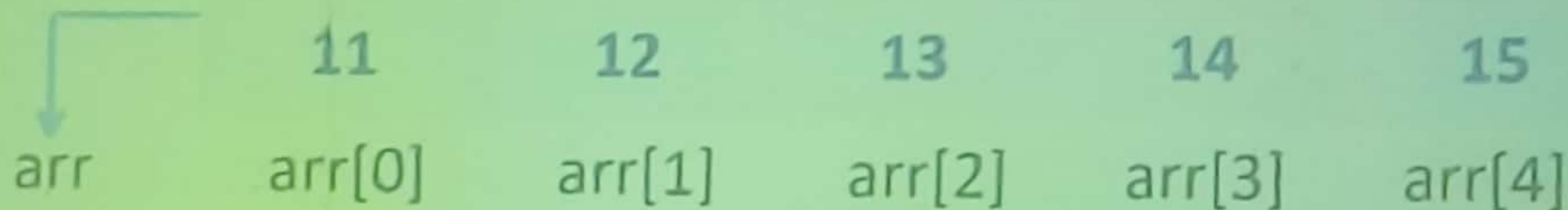
Arrow

Why Collection Framework

Lets understand about the Arrays

An array is an indexed collection of fixed number of homogeneous data elements

:



Disadvantages of Array

- Array is fixed
- Homogeneous data collection
- Non standard data structure

- Array is not implemented based on some standard Data Structure that is why the readymade method support is not available for every requirement of array.
- We have to code explicitly which increases the complexity of the program.

By Ashish G.

- Array is not implemented based on some standard Data Structure that is why the readymade method support is not available for every requirement of array.
- We have to code explicitly which increases the complexity of the program.

To overcome above problems we should go for
Collection Framework

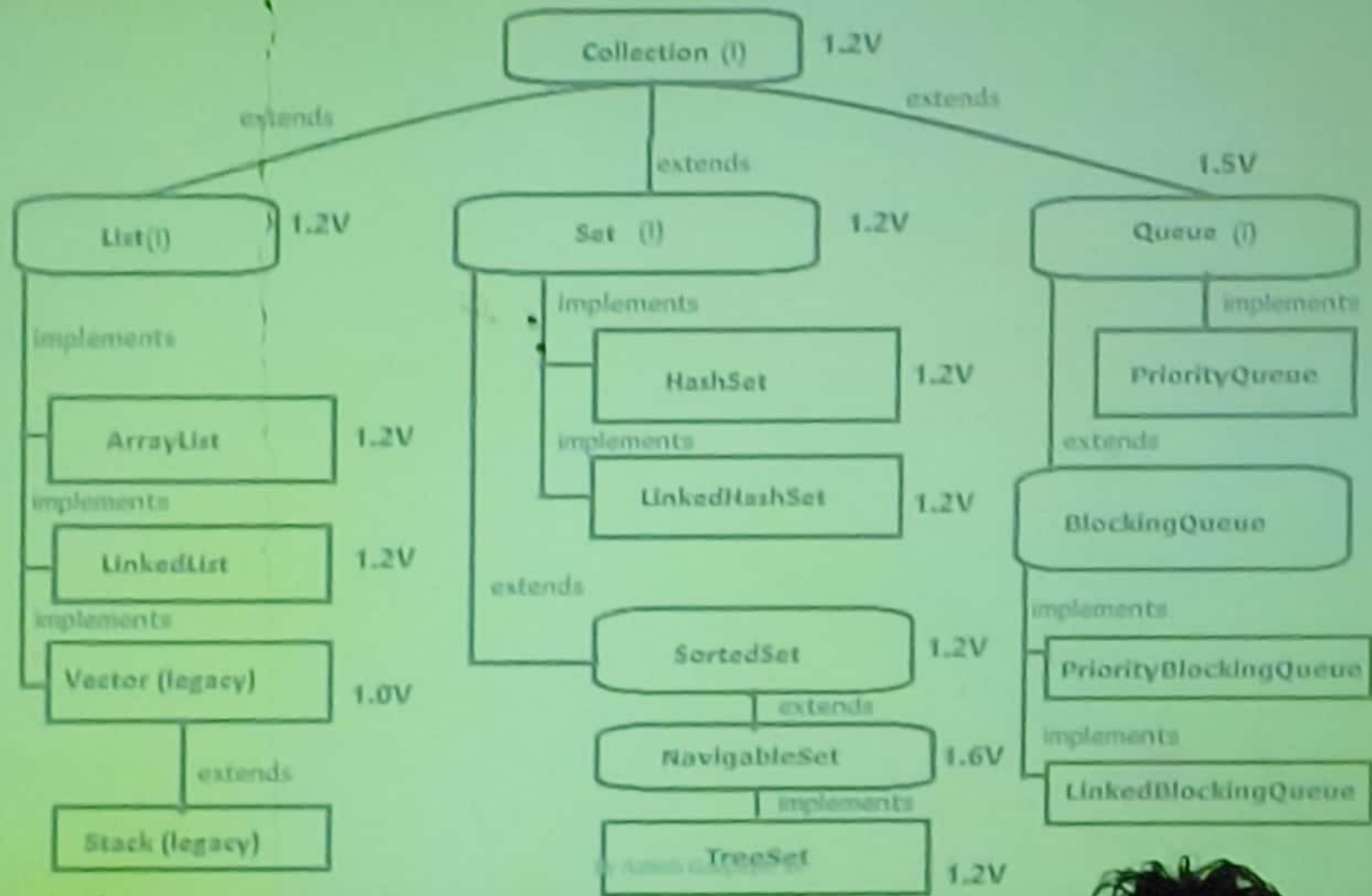
Difference between Arrays and Collections

Array	Collections
Arrays are fixed in size that is why we are not allowed to increase or decrease the size based on our requirement	Collections are grow-able in nature and based on our requirement we can increase or decrease the size
Arrays can hold both primitives as well as objects	Collections can hold only objects but are not primitive
Performance point of view arrays faster than collection	Performance point of view collections are slower than array

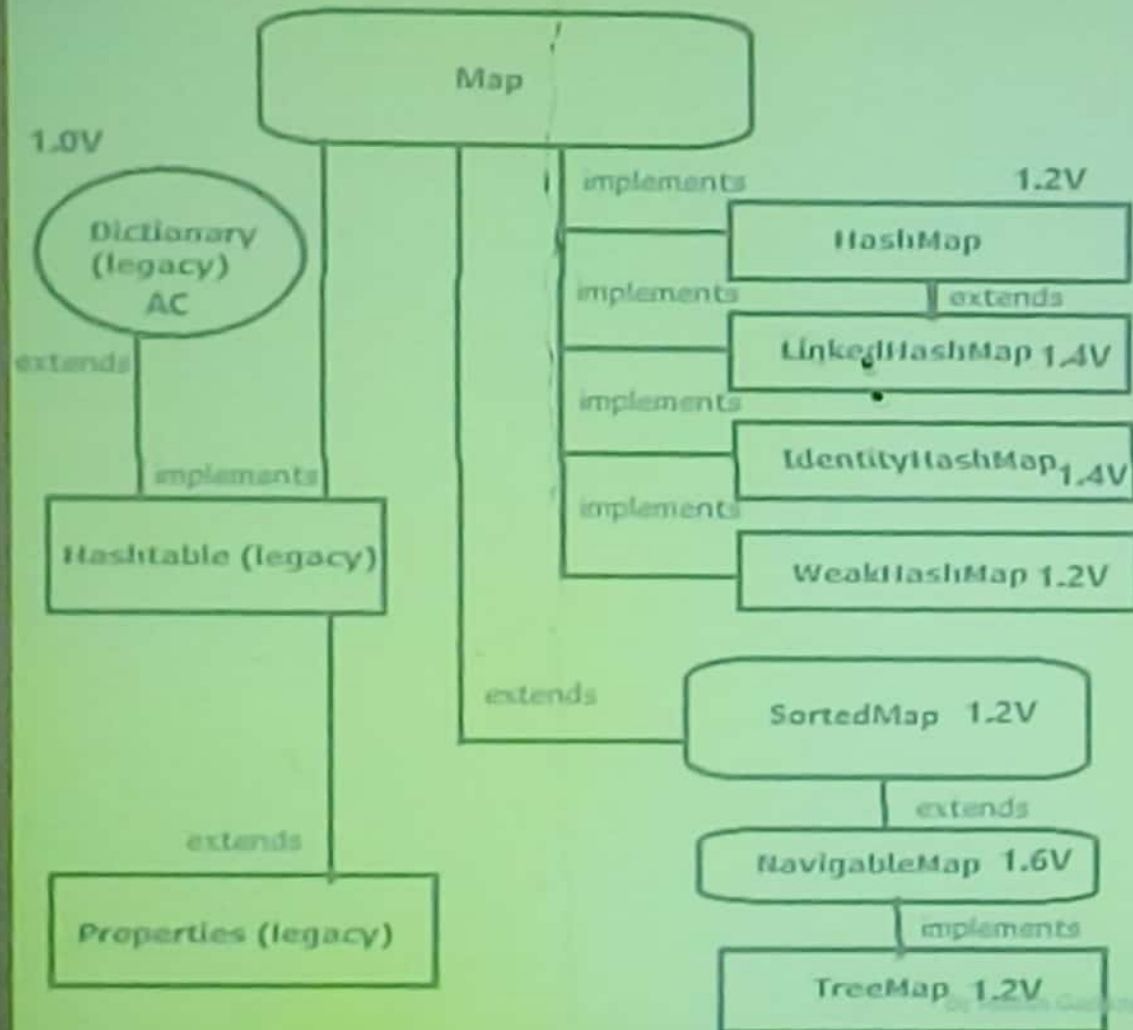
Difference between Arrays and Collections

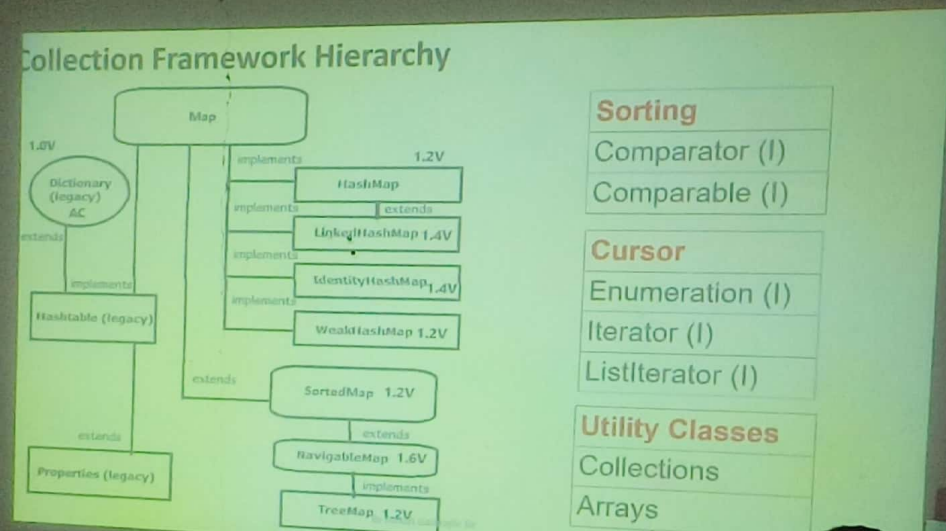
Array	Collections
Arrays can hold only homogeneous elements :	Collections can hold both homogeneous and heterogeneous elements
Memory point of view array is not recommended to use	Memory point of view collections are recommended to use
There is no data structure and readymade method support available	Every collection class implemented based on some standard data structure that is why read made method support is available

Collection Framework Hierarchy



Collection Framework Hierarchy





Sorting

Comparator (I)

Comparable (I)

Cursor

Enumeration (I)

Iterator (I)

ListIterator (I)

Utility Classes

Collections

Arrays

Collection Interface

:

If we want to represent a group of individual objects as a single entity then we should go for Collection.

Collection Interface Methods

- **boolean add(Object o):** Appends the specified element to the end of this list.
- **boolean addAll(Collection c):** Appends all of the elements in the specified collection to the end of this list.
- **boolean remove(Object o):** Removes the first occurrence of the specified element from the list
- **boolean removeAll(Collection c):** Removes from the list all of its elements that are contained in the specified collection.
- **void clear():** Removes all of the elements from the list. The list will be empty after this call returns.

Collection Interface Methods

boolean retainAll(Collection c): Removes from the list all of its elements that are not contained in the specified collection.

boolean contains(Object o): Returns true if the list contains the specified element.

boolean containsAll(Collection c): Returns true if the collection contains all of the elements in the specified collection.

boolean isEmpty(): Returns true if the list contains no elements.

int size(): Returns the number of elements in the list.

Object[] toArray(): Returns an array containing all of the elements in the list in proper sequence (from first to last element).

Iterator iterator(): Returns an iterator over the elements in the list in proper sequence.

List(I)

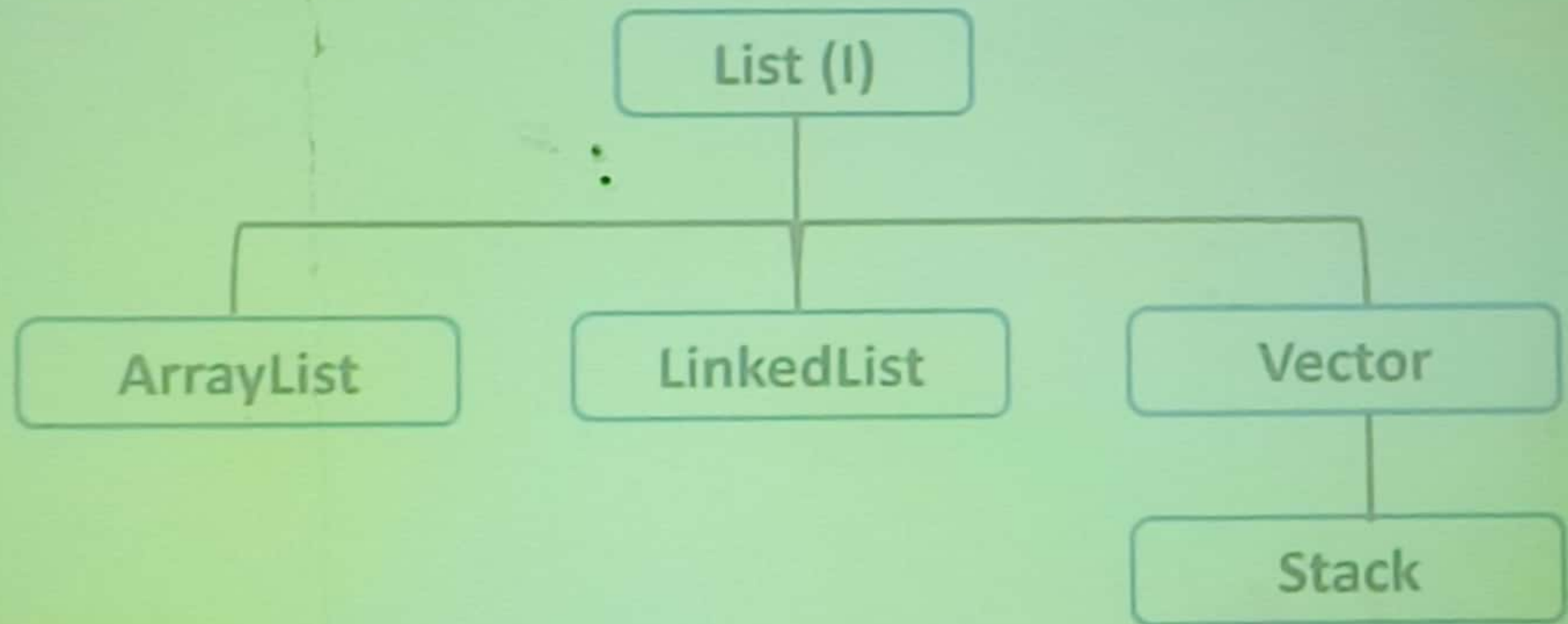
- List is a child interface of collection.
- If we want to represent a group of individual objects as a single entity where duplicates are allowed and the insertion order must be preserved then we should go for List.
- Here we can preserve insertion order via. index and we can differentiate duplicate objects by using index.

List Interface Methods

- void **add**(int index, Object o)
- boolean **addAll**(int index, Collection c)
- Object **get**(int index)
- Object **set**(int index, E element)
- Object **remove**(int index)
- int **indexOf**(Object o)
- int **lastIndexOf**(Object o)
- List **subList**(int fromIndex, int toIndex)
- ListIterator **listIterator**()
- ListIterator **listIterator**(int index)

By Ashish Gargya Sir

List Interface Implemented Classes



ArrayList

ArrayList is present in the **java.util** package. It provides us dynamic arrays in Java. Though, it may be slower than standard arrays but can be helpful in programs where lots of manipulation are needed in the array.

- Underlying data structure is Resizable array or Growable array.
- **Duplicates** are allowed.
- Insertion order **preserved**.
- Heterogeneous objects are allowed.
- **null** insertion is possible.

By Ashish Gaidapuri Sir

ArrayList - Constructors

ArrayList al=new ArrayList():

Constructs an **empty list** with an initial capacity of 10. Once ArrayList reaches max capacity then a new ArrayList object will be created with **New_Capacity** .

$$\text{New_Capacity} = (\text{Initial_Capacity} * 3/2) + 1$$

ArrayList - Constructors

ArrayList al=new ArrayList():

Constructs an **empty list** with an initial capacity of 10. Once ArrayList reaches max capacity then a new ArrayList object will be created with **New_Capacity** .

$$\text{New_Capacity} = (\text{Initial_Capacity} * 3/2) + 1$$

public ArrayList(Collection c): Constructs a **list** containing the elements of the specified collection

public ArrayList(int initialCapacity): Constructs an **empty list** with the specified initial capacity

Ex:

By: Ashish Gargadekar

ArrayList

Every collection is implemented by default **Serializable** and **Cloneable** interface.

Usually we can use collections to **hold** & **to transfer** objects from one location to another location (container). To provide support for this type of requirement, every collection class by default implements **Serializable** & **Cloneable** interface.

ArrayList and **Vector** implements **RandomAccess** interface so that any random element we can access with the same speed.

ArrayList

RandomAccess interface is present in the `java.util` package and it doesn't contain any methods, it is a **marker interface** where required ability will be provided automatically by the JVM.

ArrayList is the best choice if our frequent operation is a **retrieval** operation, because **ArrayList** implements the **RandomAccess** interface.

ArrayList is the worst choice if our frequent operation is **insertion** or **deletion** in the middle.

Difference between ArrayList and Vector

ArrayList	Vector
Methods are non synchronized	Methods are synchronized
Not thread safe , multiple thread is allowed to operate.	Only one thread is allowed to operate
Relatively performance is high	Relatively performance is low
Comes in 1.2 V, It is non legacy class	Comes in 1.0 V, It is legacy class

Get a synchronized version of **Set** object?

```
public static Set synchronizedSet(Set s)
```

Get a synchronized version of **Map** object?

```
public static Map synchronizedMap(Map m)
```

*Note: **ArrayList** is not recommended If operation is insertion and deletion in middle, to overcome above problem we should go for **LinkedList***

By Ashish Gargya for

LinkedList

The underlying Data Structure is **double linked list**.

Insertion order is **preserved**.

Duplicate objects are allowed.

Heterogeneous objects are allowed.

null insertion is possible.

LinkedList implements **Serializable** and **Cloneable** interfaces but not **RandomAccess**.

Best choice if our frequent operation is **insertion** or **deletion** in the middle.

Worst choice if our frequent operation is a **retrieval**.

LinkedList Methods

- void **add**(Object o)
- void **addFirst**(Object o)
- void **addLast**(Object o)
- Object **removeFirst**()
- Object **removeLast**()
- Object **getFirst**()
- Object **getLast**()
- int **indexOf**(Object o)

Ex

Limitations of Enumeration

We can apply **Enumeration** concept only for **legacy classes** and it is not a **universal cursor**.

We can get only **read access** but we can't perform **remove** operation.

To overcome above limitations, we should go for **Iterator**.

Iterator

- It is used to retrieve element from all collection type objects **List**, **Set** & **Queue** including **Vector**.
- By using Iterator we can perform both **read** and **remove** operations :

```
public Iterator iterator()
```

Ex:

```
Iterator itr=c.iterator()
```

Here C is any
Collection
object

Limitations of Iterator

By using **Enumeration** and **Iterator**, we can always move towards **forward direction** only but we can't move towards **backward direction**. These are single direction cursors but **not bidirectional**.

By using an **Iterator**, we can perform only **read & remove** operation & we can't perform **replacement** and **addition** of new objects.

```

1 import java.util.LinkedList;
2 class LinkedListDemo1 {
3     public static void main(String[] args) {
4         LinkedList ll=new LinkedList();
5         ll.add("Ajay");
6         ll.add(30);
7         ll.add(null);
8         System.out.println(ll);
9         ll.set(2, "Software");
10        System.out.println(ll);
11        ll.add(2, "APS");
12        System.out.println(ll);
13        ll.removeLast();
14        System.out.println(ll);
15        ll.addFirst("ccc");
16        System.out.println(ll);
17    }
18 }

```

```

1 c
1 l
a
s
s
0
b
j
e
c
t
(
2
3
}
4
5 c
l

```

Vector

Underlying Data Structure is **resizable** or **growable** array.

Insertion order is **preserved**.

Duplicate objects are allowed.

Heterogeneous objects are allowed.

null insertion is possible.

Vector

Underlying Data Structure is **resizable** or **growable** array.

Insertion order is **preserved**.

Duplicate objects are allowed.

Heterogeneous objects are allowed.

null insertion is possible.

It implements **Serializable**, **Cloneable** and **RandomAccess**.

Every method present in the **Vector** is **synchronized** hence the vector object is **thread safe**.

Worst choice if our frequent operation is **insertion** or **deletion** in the middle.

Best choice if our frequent operation is a **retrieval**

Vector - Constructors

Vector v=new Vector():

Constructs an **empty vector** with an initial capacity of 10. Once Vector reaches max capacity then a new Vector object will be created with **New_Capacity**

$$\text{New_Capacity} = \text{Initial_Capacity} * 2$$

Vector Methods

- public synchronized boolean **isEmpty()**
- public synchronized void **addElement(Object obj)**
- public synchronized Object **elementAt(int index)**
- public synchronized Object **firstElement()**
- public synchronized Object **lastElement()**
- public synchronized void **insertElementAt(Object obj, int index)**
- public synchronized void **setElementAt(Object obj, int index)**
- public synchronized boolean **removeElement(Object obj)**

By Ashish Gadgil Sir

Vector Methods

- public synchronized boolean **isEmpty()**
- public synchronized void **addElement(Object obj)**
- public synchronized Object **elementAt(int index)**
- public synchronized Object **firstElement()**
- public synchronized Object **lastElement()**
- public synchronized void **insertElementAt(Object obj, int index)**
- public synchronized void **setElementAt(Object obj, int index)**
- public synchronized boolean **removeElement(Object obj)**

By Ashish Gaudipalle Sr

Stack

It is a child class of **Vector**.

It is a specially designed class for **LIFO (Last In First Out)**.

When a Stack is first created, it contains no element.

In this class the last element inserted is accessed first.

Stack Methods

- **public boolean empty():** Tests if this stack is empty.
- **public synchronized Object peek():** Looks at the object at the top of this stack without removing it from the stack.
- **public synchronized Object pop():** Removes the object at the top of this stack and returns that object.
- **public Object push(Object item):** Pushes an object item onto the top of this stack.
- **public synchronized int search(Object o):** Returns the offset from top of the stack else return -1.

Stack Methods

- **public boolean empty():** Tests if this stack is empty.
- **public synchronized Object peek():** Looks at the object at the top of this stack without removing it from the stack.
- **public synchronized Object pop():** Removes the object at the top of this stack and returns that object.
- **public Object push(Object item):** Pushes an object item onto the top of this stack.
- **public synchronized int search(Object o):** Returns the offset from top of the stack else return -1.

```
1 import java.util.*;
2 class StackDemo1 {
3     public static void main(String[] args)
4     {
5         Stack<String> s = new Stack<String>();
6         s.push("Raj"); //insert the data top of the stack
7         s.push("Pravin"); //insert the data top of the stack
8         s.push("Manoj");
9         System.out.println(s);
10        System.out.println(s.search("Manoj")); //1 last added objec
11        System.out.println(s.size());
12        System.out.println(s.peek()); //to return last element of
13        s.pop(); //remove the data top of the stack
14        System.out.println(s);
15        System.out.println(s.isEmpty());
16        s.clear();
17        System.out.println(s.isEmpty());
18    }
}
```

- If we want to get objects one by one from the **Collection** then we should go for a cursor.

There are three types of cursor

- Enumeration
- Iterator
- ListIterator

```

1 //Enumeration example
2 import java.util.*;
3 class CursorDemo{
4     public static void main(String[] args) {
5         Vector v=new Vector();
6         int sum=0;
7         for(int i=0;i<10;i++)
8         {
9             v.addElement(i);
10        }
11        System.out.println(v);
12        Enumeration e=v.elements();
13        while(e.hasMoreElements())
14        {
15            Integer in =(Integer)e.nextElement();
16            if(in%2==0){
17                System.out.println(in);
18                sum+=in;

```

Limitations of Iterator

By using **Enumeration** and **Iterator**, we can always move towards **forward direction** only but we can't move towards **backward direction**. These are single direction cursors but **not bidirectional**.

By using an **Iterator**, we can perform only **read & remove** operation & we can't perform **replacement** and **addition** of new objects.

To overcome above limitations, we should go for **ListIterator**.

ListIterator

It is a **bidirectional** cursor and used to retrieve elements only from List implemented collection objects like **ArrayList**, **Vector**, **Stack**, **LinkedList**.

It is a sub interface of **Iterator**. Hence we can access all methods of **Iterator**.

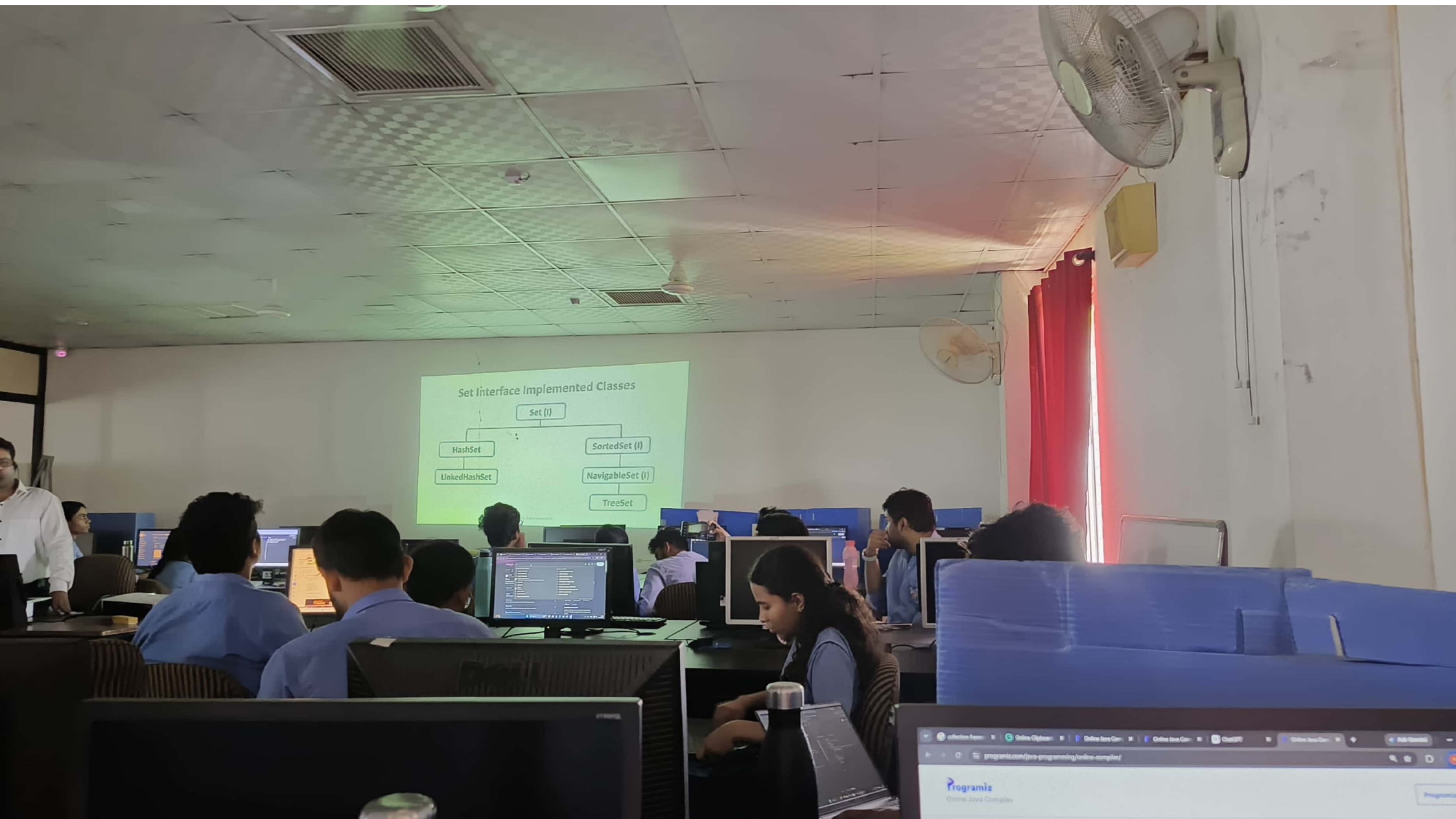
By using **ListIterator**, we can perform **replacement** & the **addition** of new objects in addition to **read & remove** operation.

ListIterator Methods

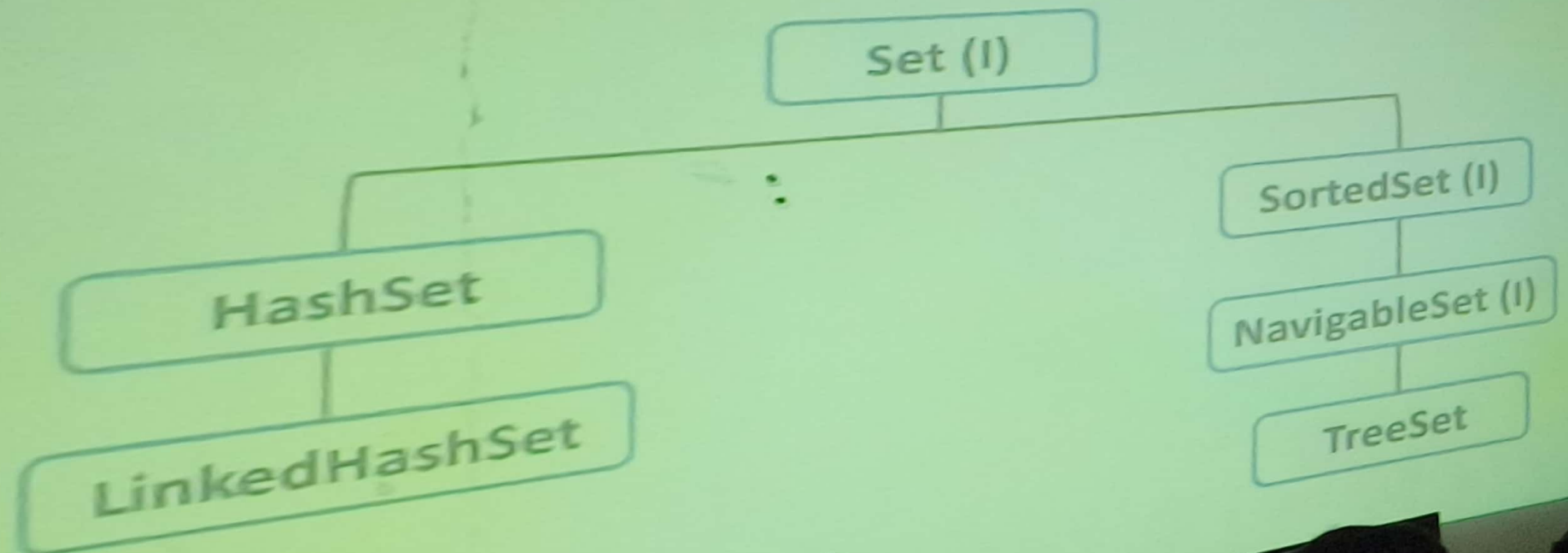
- public boolean **hasNext()**
- public Object **next()**
- public boolean **hasPrevious()**
- public Object **previous()**
- public void **remove()**
- public void **add(Object o)**
- public void **set(Object o)**

Comparison of three Cursors

Property	Enumeration	Iterator	ListIterator
Applicable for	Only for legacy classes	Any Collection classes	Only for List classes
Movement	Only forward direction(single direction cursor)	Only forward direction(single direction cursor)	Both forward and backward direction (bidirectional)
Accessibility	Only read access	Both read and remove	Read, remove, replace and addition of new object
How to get it?	By using elements() method of Vector	By using iterator() method of Collection	By using listIterator() method() of List interface
Methods	2 methods	3 methods	9 methods
Is it legacy	Yes (1.0v)	No (1.2v)	No (1.2v)



Set Interface Implemented Classes



HashSet
The underlying data structure is **Hashtable**.
Duplicate objects are not allowed.
Insertion order is **not preserved** and it is based on **HashCode** of the objects.
null insertion is possible only once.
Heterogeneous objects are allowed.
Implements **Serializable**, **Cloneable** but not **RandomAccess** interface.
HashSet is the best choice if our frequent operation is **search** operation.

HashSet - Constructors

public HashSet(): Constructs a new empty set with default initial capacity 16 and fill ratio (load factor) is 0.75

public HashSet(Collection c): The new HashSet is created with a default load factor (0.75) and an initial capacity

public HashSet(int initialCapacity): Constructs a new empty set with specified initial capacity

public HashSet(int initialCapacity, float loadFactor): Constructs a new, empty set with specified initial capacity and the specified load factor

LinkedHashSet

It is a child class of **HashSet**.

It is exactly the same as **HashSet** (including **constructors** & **methods**) except the following differences.

HashSet	LinkedHashSet
Underlying Data Structure is Hashtable	Underlying Data Structure is LinkedList + Hashtable
Insertion order is not preserved	Insertion order is preserved
It is in java version 1.2V	It is in java version 1.4V

LinkedHashSet

We can use LinkedHashSet to develop **cache based applications** where

- Duplicates are not allowed
- Insertion order is preserved

Typically a caching module works like this

Step 1: Check if value is there in Cache, if yes return

Step 2: If value is null in **Step 1** then make the API call to Database to get the value

Step 3: Return the value to the User

Step 4: Store it in the Cache for next time usage

TreeSet

Underlying Data Structure is a **balanced tree**.

Duplicates are not allowed.

All objects will be inserted based on some sorting order, It may be **default natural sorting** order or **customize sorting** order.

Heterogeneous objects are not allowed.

null insertion is possible (only once).

TreeSet implements **Serializable** & **Cloneable** but not **RandomAccess**.